

DSA EXPERIMENT 3:Bubble sort, Selection sort, Insertion, Quick and Merge sort.

NAME:Rushita Kiran Sontakke
BATCH:B1

PRN:B25ET1051
CLASS:SY1

Bubblesort

```
#include <stdio.h>
void bubblesort(int arr[], int n) {
    int temp;

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int arr[] = {98, 65, 78, 43, 32, 82};
    int n = 6;
    int i;

    printf("Original array: ");
    for (i=0;i<6;i++)
        printf("%d ",arr[i]);

    bubblesort(arr, n);

    printf("\nSorted array:");
    for (i=0;i<6;i++)
        printf("%d ",arr[i]);

    return 0;
}
```

Output

Original array: 98 65 78 43 32 82

Sorted array: 32 43 65 78 82 98

=== Code Execution Successful ===

Selection sort

```
#include<stdio.h>

void selectionsort(int arr[], int n) {
    int minindex, temp;

    for (int i = 0; i < n - 1; i++) {
        minindex = i;

        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minindex]) {
                minindex = j;
            }
        }

        temp = arr[i];
        arr[i] = arr[minindex];
        arr[minindex] = temp;
    }
}

int main() {
    int arr[] = {98, 65, 78, 43, 32, 82};
    int n = 6;
    int i;

    printf("Original array: ");{
    for (i=0;i<6;i++)
    printf("%d ",arr[i]);}

    selectionsort(arr, n);

    printf("\nSorted array:");{
    for (i=0;i<6;i++)
    printf("%d ",arr[i]);}
```

```
return 0;
```

Output

```
Original array: 98 65 78 43 32 82
```

```
Sorted array: 32 43 65 78 82 98
```

```
=== Code Execution Successful ===
```

```
}
```

Insertion sort

```
#include <stdio.h>
```

```
void insertionsort(int arr[], int n)
```

```
{
```

```
    int i, j, key;
```

```
    for (i = 1; i < n; i++)
```

```
    {
```

```
        key = arr[i];
```

```
        j = i - 1;
```

```
        while (j >= 0 && arr[j] > key)
```

```
        {
```

```
            arr[j + 1] = arr[j];
```

```
            j--;
```

```
        }
```

```
        arr[j + 1] = key;
```

```
    }
```

```
}
```

```
int main()
```

```
{
```

```
    int arr[] = {79, 90, 67, 52, 88};
```

```
    int n = 5;
```

```
    int i;
```

```
    printf("Original array: ");
```

```
    for (i = 0; i < n; i++)
```

```
        printf("%d ", arr[i]);
```

```
    insertionsort(arr, n);
```

```
printf("\nSorted array: ");
for (i = 0; i < n; i++)
    printf("%d ", arr[i]);

return 0;
}
```

Output

```
Original array: 79 90 67 52 88
Sorted array: 52 67 79 88 90

=== Code Execution Successful ===
```

Quick Sort

```
#include <stdio.h>
```

```
void swap(int *a, int *b)
```

```
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

```
int partition(int arr[], int low, int high)
```

```
{
    int pivot = arr[high];
    int i = low - 1;
    int j;

    for (j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
}
```

```

    }
}

swap(&arr[i + 1], &arr[high]);

return i + 1;
}

void quicksort(int arr[], int low, int high)
{
    if (low < high)
    {
        int p = partition(arr, low, high);

        quicksort(arr, low, p - 1);
        quicksort(arr, p + 1, high);
    }
}

int main()
{
    int arr[100];
    int n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);

    for (i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Original array: ");
    for (i = 0; i < n; i++)

```

```

    {
        printf("%d ", arr[i]);
    }

    quicksort(arr, 0, n - 1);

    printf("\nSorted array: ");
    for (i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}

```

Output

```

Enter number of elements: 5
Enter 5 elements:
88 12 52 78 9
Original array: 88 12 52 78 9
Sorted array: 9 12 52 78 88

=== Code Execution Successful ===

```

MERGE SORT

```
#include <stdio.h>
```

```

void Merge(int a[], int l, int mid, int h)
{
    int i = l;
    int j = mid + 1;
    int k = l;
    int b[100];

    while (i <= mid && j <= h)

```

```

{
    if (a[i] < a[j])
    {
        b[k] = a[i];
        i++;
    }
    else
    {
        b[k] = a[j];
        j++;
    }
    k++;
}

while (i <= mid)
{
    b[k] = a[i];
    i++;
    k++;
}

while (j <= h)
{
    b[k] = a[j];
    j++;
    k++;
}

for (i = l; i <= h; i++)
{
    a[i] = b[i];
}
}

void Mergesort(int a[], int l, int h)
{
    if (l < h)
    {
        int mid = (l + h) / 2;

```

```

        Mergesort(a, l, mid);
        Mergesort(a, mid + 1, h);

        Merge(a, l, mid, h);
    }
}

int main()
{
    int a[100], n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }
    printf("\nOriginal array:");
    for(int i=0;i<n;i++)
    {
        printf("%d ", i);
    }


    Mergesort(a, 0, n - 1);

    printf("\nSorted array: ");
    for (int i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }

    return 0;
}

```

Output

Enter number of elements: 5

Enter elements: 50 10 30 40 20

Original array:0 1 2 3 4

Sorted array: 10 20 30 40 50

=== Code Execution Successful ===